2_2_data_analysis_OASIS

2020-09-09

Custom functions for plotting

```
getHistogram <- function(data, x, xlab, ylab, n_label_x, n_label_y, label, mean, median, pvalue,
  ylim = NULL, xlim = NULL, n_digits = 2, n_decimal_p = 3, n_sigmap = 2){
  if(is.null(ylim)){ylim <- c(0, Inf)}
  if(is.null(xlim)){xlim <- c(-Inf, Inf)}
  ggplot(data, aes(x = .data[[x]])) + geom_histogram(bins = 30, fill = "lightgrey", color = "black")
    + coord_cartesian(ylim = ylim, xlim = xlim) + geom_vline(xlim = xlim, linetype = "dashed")
    + ylab(ylab) + xlab(xlab)
    + theme_bw()
    + theme(
      axis.text = element_text(size = 7.5),
      axis.title = element_text(size = 8),
      plot.title = element_text(size = 8),
      legend.text = element_text(size = 7.5),
      legend.title = element_text(size = 8),
      legend.key.size = unit(0.5, "cm"),
      legend.position = "bottom",
      strip.text = element_text(size = 8),
      strip.background = element_blank(),
      panel.grid.minor = element_blank(),
      panel.grid.major = element_blank)
    + annotate("text", x = n_label_x, y = n_label_y, label = n_label)
    + annotate("text", x = mean, y = n_label_y, label = "mean")
    + annotate("text", x = median, y = n_label_y, label = "median")
    + annotate("text", x = n_label_x, y = n_label_y - n_digits*0.5, label = round(mean, n_digits))
    + annotate("text", x = n_label_x, y = n_label_y - n_digits*0.5, label = round(median, n_digits))
    + annotate("text", x = n_label_x, y = n_label_y - n_digits*1.0, label = round(pvalue, n_decimal_p, n_sigmap))
    + ggtitle(paste(label, "n = ", length(data[[x]]), sep = ""))}

getHistogramGroup <- function(data, x, xlab, ylab, n_label_x, n_label_y, group, mean, median,
  pvalue, ylim = NULL, xlim = NULL, n_digits = 2, n_decimal_p = 3, n_sigmap = 2){
  if(is.null(ylim)){ylim <- c(0, Inf)}
  if(is.null(xlim)){xlim <- c(-Inf, Inf)}
  ggplot(data, aes(x = .data[[x]], fill = .data[[group]])) + geom_histogram(bins = 30, color = "black")
    + coord_cartesian(ylim = ylim, xlim = xlim) + geom_vline(xlim = xlim, linetype = "dashed")
    + ylab(ylab) + xlab(xlab)
    + theme_bw()
    + theme(
      axis.text = element_text(size = 7.5),
      axis.title = element_text(size = 8),
      plot.title = element_text(size = 8),
      legend.text = element_text(size = 7.5),
      legend.title = element_text(size = 8),
      legend.key.size = unit(0.5, "cm"),
      legend.position = "bottom",
      strip.text = element_text(size = 8),
      strip.background = element_blank(),
      panel.grid.minor = element_blank(),
      panel.grid.major = element_blank)
    + annotate("text", x = mean, y = n_label_y, label = "mean")
    + annotate("text", x = median, y = n_label_y, label = "median")
    + annotate("text", x = n_label_x, y = n_label_y - n_digits*0.5, label = round(mean, n_digits))
    + annotate("text", x = n_label_x, y = n_label_y - n_digits*0.5, label = round(median, n_digits))
    + annotate("text", x = n_label_x, y = n_label_y - n_digits*1.0, label = round(pvalue, n_decimal_p, n_sigmap))
    + ggtitle(paste(label, "n = ", length(data[[x]]), sep = ""))}
```

Custom function for Mahalanobis distance

```
getMahalanobis <- function(mydata, vc, cols){ mm <- as.matrix(mydata[,cols], ncol=length(cols))
  for(i in 1:length(cols)){ mm[,i] <- mm[,i] - vc[i]}
  return(diag(mm %*% solve(cov(mm[,cols]), tol=1e-20) %*% t(mm)))}
```

Custom function for SEM table

```
getSEMtable <- function(fit, ci = TRUE, digits = 3, sort = "lhs", rm.eq = TRUE, rm.ov = FALSE){
  if(class(fit) == "lavaan"){ parTable <- parTable(fit)
    if(ci == TRUE){ ci <- parameterEstimates(fit, ci=TRUE, level=0.95, boot.ci.type="bca.simple")
    parTable <- merge(parTable, ci, by=c("lhs","op","rhs"))}
    if(rm.eq == TRUE){ parTable <- parTable[parTable$op != "==",]}
    if(rm.ov == TRUE){
      latent <- unique(parTable[parTable$op == "=~",]$lhs)
      parTable <- parTable[parTable$op == "=~" | parTable$rhs %in% latent,]}
    if(sort == "lhs"){ parTable <- parTable[order(parTable$lhs),]}
    if(sort == "rhs"){ parTable <- parTable[order(parTable$rhs),]}
    if(digits > 0){ parTable[,c("est","se","ci.lower","ci.upper")] <- round(parTable[,c("est","se","ci.lower","ci.upper")], digits)}
    parTable[,c("stat","pvalue")] <- round(parTable[,c("stat","pvalue")], 2)
    rownames(parTable) <- NULL
    return(parTable)}
  else{ print("Error: class(fit) != lavaan")}}
```

Custom function for SEM plots

```
getSEMplot <- function(pathdata, layout, title, ylim = c(-1,1), xlim = c(-1,1), edge.label.cex = 0.75){
  qgraph(pathdata, layout=layout, title=title, ylim=ylim, xlim=xlim, edge.color="black",
    edge.labels=TRUE, edge.label.cex=edge.label.cex, edge.label.position=0.5, vsize=10, esize=10,
    asize=5, mar=c(5,5,5,5), nodeNamesFrom="name")}
```

Read in data

```
mydata.longer <- read.csv("data_processed/longer_check_and_clean/longer_qc_dataset.csv")
mydata.wide <- read.csv("data_processed/wide_check_and_clean/wide_qc_dataset.csv")
mydata.longer.complete <- read.csv("data_processed/longer_complete/longer_qc_complete.csv")
mydata.longer.complete.imp <- read.csv("data_processed/longer_complete/longer_qc_complete_imp.csv")
mydata.wide.complete.imp <- read.csv("data_processed/wide_complete/wide_qc_complete_imp.csv")
```

CROSS-SECTIONAL ANALYSIS

Cross-sectional analysis of the sample.

```
library(psych)
library(ggplot2)
```

```
## Loading required package: grid
```

```
library(kableExtra)
```

```
## Warning: package 'kableExtra' was built under R version 4.0.2
```

```
## Warning: package 'dplyr' was built under R version 4.0.2
```

```
## Warning: package 'tidyr' was built under R version 4.0.2
```

```
## Warning: package 'stringr' was built under R version 4.0.2
```

```
## Warning: package 'forcats' was built under R version 4.0.2
```

```
## Warning: package 'lubridate' was built under R version 4.0.2
```

```
## Warning: package 'DT' was built under R version 4.0.2
```

```
## Warning: package 'ggsurvplot' was built under R version 4.0.2
```

```
## Warning: package 'survminer' was built under R version 4.0.2
```

```
## Warning: package 'ggpubr' was built under R version 4.0.2
```

```
## Warning: package 'ggfortify' was built under R version 4.0.2
```

```
## Warning: package 'survival' was built under R version 4.0.2
```

```
## Warning: package 'Hmisc' was built under R version 4.0.2
```

```
## Warning: package 'lattice' was built under R version 4.0.2
```

```
## Warning: package 'Rcpp' was built under R version 4.0.2
```

```
## Warning: package 'RcppArmadillo
```

Thickness Change Rate

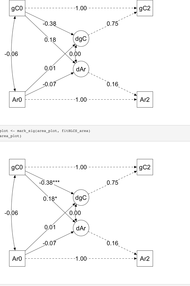
Surface Area Change Rate

```
# Thickness change rate
# Surface area change rate
# Baseline thickness
# Baseline surface area
# Age
# Sex
# Cortical parcellation
# Subcortical volumes
# Total intracranial volume
# Scanner
# Site
# Scanner and site
```

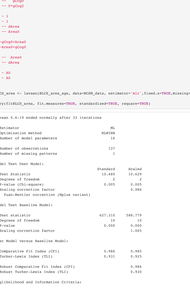
```
boosting_mse_tr = GradientBoosting_mse_tr * TRAIN;
boosting_mse_cv = GradientBoosting_mse_cv * CV_FOLDS;
boosting_mse_test = GradientBoosting_mse_test * TEST;

# Ridge
ridge_mse_tr = Ridge_mse_tr * TRAIN; # train set
ridge_mse_cv = Ridge_mse_cv * CV_FOLDS; # cross-validation
ridge_mse_test = Ridge_mse_test * TEST; # test set

# Lasso
lasso_mse_tr = Lasso_mse_tr * TRAIN; # train set
lasso_mse_cv = Lasso_mse_cv * CV_FOLDS; # cross-validation
lasso_mse_test = Lasso_mse_test * TEST; # test set

# Elastic Net
elastic_mse_tr = Elastic_mse_tr * TRAIN; # train set
elastic_mse_cv = Elastic_mse_cv * CV_FOLDS; # cross-validation
elastic_mse_test = Elastic_mse_test * TEST; # test set

# Support Vector Regression
svr_mse_tr = Support_Vector_Regression_mse_tr * TRAIN; # train set
svr_mse_cv = Support_Vector_Regression_mse_cv * CV_FOLDS; # cross-validation
svr_mse_test = Support_Vector_Regression_mse_test * TEST; # test set

# Decision Tree
decision_tree_mse_tr = Decision_Tree_mse_tr * TRAIN; # train set
decision_tree_mse_cv = Decision_Tree_mse_cv * CV_FOLDS; # cross-validation
decision_tree_mse_test = Decision_Tree_mse_test * TEST; # test set

# Random Forest
random_forest_mse_tr = Random_Forest_mse_tr * TRAIN; # train set
random_forest_mse_cv = Random_Forest_mse_cv * CV_FOLDS; # cross-validation
random_forest_mse_test = Random_Forest_mse_test * TEST; # test set

# Gradient Boosting
boosting_mse_tr = GradientBoosting_mse_tr * TRAIN; # train set
boosting_mse_cv = GradientBoosting_mse_cv * CV_FOLDS; # cross-validation
boosting_mse_test = GradientBoosting_mse_test * TEST; # test set
```

```
##
##  Standard errors                                   Sandwich
##  Information based                                 Observed
##  Observed information based on                     Hessian
##
## Latent Variables:
##                   Estimate Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  dfbvs =~
##    dfgbvs            1.000                               0.161    0.161
##  dfgbvs =~
##    gfgqt             2.000                               0.194    0.751
##
## Regressions:
##                   Estimate Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  Areal ~
##    hcesd             1.000                               1.000    1.000
##    gfgqt             1.000                               1.000    1.000
##  dfbvs ~
##    gfgqt             0.030   0.014   2.062   0.039   0.184    0.184
##    hcesd             0.012   0.012   0.953   0.340   0.071    0.071
##  dfgbvs ~
##    hcesd             0.208   0.054   3.158   0.474   0.211    0.211
##    gfgqt             0.282   0.042  -6.566   0.000  -1.256   -1.256
##
## Covariances:
##                   Estimate Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  Areal ~~
##    gfgqt            -0.057   0.066  -0.875   0.500  -0.057   -0.057
##  dfbvs ~~
##    dfgbvs            0.000   0.008   0.043   0.965   0.003    0.003
##
## Intercepts:
##                   Estimate Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    hcesd             0.000   0.000       NA       NA   0.000    0.000
##    dfgbvs            0.000   0.000       NA       NA   0.000    0.000
##    gfgqt             0.000   0.085   0.000   1.000   0.000    0.000
##    dfbvs             0.000   0.000       NA       NA   0.000    0.000
##    dfbvs             -0.000   0.013  -0.000   1.000  -0.000   -0.000
##    dfgbvs            -0.000   0.085  -0.000   1.000  -0.000   -0.000
##
## Variances:
##                   Estimate Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    hcesd             0.000   0.000       NA       NA   0.000    0.000
##    dfgbvs            0.441   0.066   7.341   0.000   0.459    0.459
##    gfgqt             0.000   0.000   1.118   0.997   0.000    0.000
##    dfbvs             0.000   0.000       NA       NA   0.000    0.000
##    dfbvs             0.025   0.005   5.514   0.000   0.968    0.968
##    dfgbvs            0.043   0.115   0.256   0.000   0.951    0.200
##
## R-Squares:
##                   Estimate
##    hcesd             1.000
##    dfgbvs            0.541
##    gfgqt             1.000
##    dfbvs             0.032
##
## Defined Parameters:
##                   Estimate
##    hcesd             1.000
##    dfgbvs            0.000
##    gfgqt             0.000
##    dfbvs             0.000
##
## R-Square:
##                   Estimate
##    hcesd             1.000
##    dfgbvs            0.541
##    gfgqt             1.000
##    dfbvs             0.032
##
## Defined Parameters:
##                   Estimate
##    hcesd             1.000
##    dfgbvs            0.000
##    gfgqt             0.000
##    dfbvs             0.000
##
## Fit Indices:
##                   Estimate  Std.err  z-value  P(>|z|)
##  Chi-square        0.000    0.000       NA       NA
##  Df                1.000    0.000       NA       NA
##  Pr(>Chi-sq)       0.000    0.000       NA       NA
##  Chi-square ratio  0.000    0.000       NA       NA
##  Chi-square null   2.242    0.029   76.184    0.000
##  Df null           1.000    0.000       NA       NA
##  Pr(>Chi-sq) null  0.000    0.000       NA       NA
##  Fit Indices:
##                   Estimate  Std.err  z-value  P(>|z|)
##  Chi-square        0.000    0.000       NA       NA
##  Df                1.000    0.000       NA       NA
##  Pr(>Chi-sq)       0.000    0.000       NA       NA
##  Chi-square ratio  0.000    0.000       NA       NA
##  Chi-square null   2.242    0.029   76.184    0.000
##  Df null           1.000    0.000       NA       NA
##  Pr(>Chi-sq) null  0.000    0.000       NA       NA
```

```
area_plot <- semPlot::semPaths(fit_SCAR_area,
                               layout = "tree",
                               edge.color = "black",
                               rotation = 2,
                               edge.label.cex = 1.5,
                               label.cex = 1.2,
                               sizeMan = 8,
                               sizeLat = 8,
                               nCharNodes = 0,
                               nCharEdges = 0,
                               residuals = FALSE,
                               edge.color = "black")
```



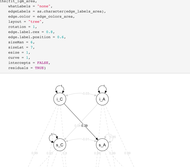

```
area_plot <- mark_sig(area_plot, fit_SCAR_area)
plot(area_plot)
```




```
SCAR_area_ego <- lavsem(SCAR_area_ego, data=BCSR_data, estimator='ML',fixed.x=TRUE,missing='fim
l')
summary(fitSCAR_area, fit.measures=TRUE, standardized=TRUE, rsquare=TRUE)
```

```
## lavsem 0.6-19 ended normally after 31 iterations
##
##  Estimator                                         ML
##  Optimization method                           NLMINB
##  Number of model parameters                        14
##
##  Number of observations                           127
##  Number of missing patterns                         1
##
## Model Test User Model:
##                                               Standard      Scaled
##  Test statistic                                 10.485      10.429
##  Degrees of freedom                                 15          15
##  P-value (Chi-square)                            0.005       0.005
##  Scaling correction factor                               0.984
##    Yuan-Bentler correction (Mplus variant)
##
## Model Test Baseline Model:
##
##  Test statistic                               427.110     588.779
##  Degrees of freedom                                 21          21
##  P-value                                         0.000       0.000
##  Scaling correction factor                               0.725
##
## User Model versus Baseline Model:
##
##  Comparative Fit Index (CFI)                     0.986       0.985
##  Tucker-Lewis Index (TLI)                        0.921       0.923
##
##  Robust Comparative Fit Index (CFI)              0.986
##  Robust Tucker-Lewis Index (TLI)                 0.930
##
## Loglikelihood and Information Criteria:
##
##  Loglikelihood user model (H0)                -467.156    -467.156
##  Scaling correction factor                               1.565
##      for the MLR correction
##  Loglikelihood unrestricted model (H1)        -461.914    -461.914
##  Scaling correction factor                               1.839
##      for the MLR correction
##
##  Akaike (AIC)                                 944.312     944.312
##  Bayesian (BIC)                               989.124     989.124
##  Sample-size adjusted Bayesian (SABIC)        912.616     912.616
##
## Root Mean Square Error of Approximation:
##
##  RMSEA                                          0.176       0.177
##  90 Percent confidence interval - lower         0.071       0.071
##  90 Percent confidence interval - upper         0.287       0.289
##  P-value H_0: RMSEA <= 0.050                    0.012       0.011
##  P-value H_0: RMSEA >= 0.080                    0.863       0.854
##
##  Robust RMSEA                                           0.177
##  90 Percent confidence interval - lower                 0.086
##  90 Percent confidence interval - upper                 0.285
##  P-value H_0: Robust RMSEA <= 0.050                     0.014
##  P-value H_0: Robust RMSEA >= 0.080                     0.859
##
## Standardized Root Mean Square Residual:
##
##  SRMR                                           0.074       0.074
##
## Parameter Estimates:
##
##  Standard errors                                   Sandwich
##  Information                                       Observed
##  Observed information based on                     Hessian
##
## Latent Variables:
##                   Estimate  Std.err  z-value  P(>|z|)   Std.lv  Std.all
##  dfbvs =~
##    dfgbvs            1.000                               0.160    0.160
##  dfgbvs =~
##    gfgqt             1.000                               0.752    0.759
##
## Regressions:
##                   Estimate  Std.err  z-value  P(>|z|)   Std.lv  Std.all
##  Areal ~
##    hcesd             1.000                               1.000    1.000
##    gfgqt             1.000                               1.000    1.000
##  dfbvs ~
##    gfgqt             0.017   0.014   1.219   0.223   0.185    0.185
##    hcesd             0.013   0.012   1.047   0.296   0.081    0.081
##  dfgbvs ~
##    hcesd             0.208   0.054   3.195   0.469   0.210    0.210
##    gfgqt             -0.290   0.061  -4.766   0.000  -1.386   -1.386
##
##  Covariances:
##                   Estimate  Std.err  z-value  P(>|z|)   Std.lv  Std.all
##  Areal ~~
##    gfgqt             -0.057   0.066  -0.875   0.500   -0.057   -0.057
##  dfbvs ~~
##    dfgbvs            -0.001   0.008  -0.114   0.909   -0.009   -0.009
##
## Intercepts:
##                   Estimate  Std.err  z-value  P(>|z|)   Std.lv  Std.all
##    hcesd             0.000   0.000       NA       NA   0.000    0.000
##    dfgbvs            0.000   0.000       NA       NA   0.000    0.000
##    gfgqt             0.000   0.085   0.000   1.000   0.000    0.000
##    dfbvs             0.000   0.012   0.000   1.000   0.000    0.000
##    dfbvs             -0.000   0.005  -0.000   1.000  -0.000   -0.000
##
## Variances:
##                   Estimate  Std.err  z-value  P(>|z|)   Std.lv  Std.all
##    hcesd             0.000   0.000       NA       NA   0.000    0.000
##    dfgbvs            0.440   0.065   7.424   0.000   0.450    0.450
##    gfgqt             0.000   0.000   1.118   0.997   0.000    0.000
##    dfbvs             0.000   0.000       NA       NA   0.000    0.000
##    dfbvs             0.023   0.006   4.038   0.000   0.095    0.095
##    dfgbvs            0.043   0.115   0.256   0.000   0.951    0.200
##
## R-Squares:
##                   Estimate
##    hcesd             1.000
##    dfgbvs            0.550
##    gfgqt             1.000
##    dfbvs             0.105
```

```
lgm_area_model <-
  '
  i_gfgq =~ 1*gfgq0 + 1*gfgq1 + 1*gfgq2
  s_gfgq =~ 0*gfgq0 + 1*gfgq1 + 2*gfgq2

  i_area =~ 1*hcesd + 1*hcesd + 1*hcesd
  s_area =~ 0*hcesd + 1*hcesd + 2*hcesd

  i_gfgq ~ i
  s_gfgq ~ i
  i_area ~ i
  s_area ~ i

  i_gfgq ~~ i_area
  s_gfgq ~~ s_area

  s_area ~ i_gfgq
  s_gfgq ~ s_area

  gfgq0 ~~ 0.01*gfgq0
  gfgq1 ~~ 0.01*gfgq1
  gfgq2 ~~ 0.01*gfgq2

  hcesd ~~ 0.01*hcesd
  hcesd ~~ 0.01*hcesd
  hcesd ~~ 0.01*hcesd
  '
fit_lgm_area_model <- growth(lgm_area_model, data = BCSR_data, missing = 'fiml')
summary(fit_lgm_area, fit.measures = TRUE, standardized = TRUE, rsquare = TRUE)
```

```
## lavaan 0.6-19 ended normally after 37 iterations
##
##  Estimator                                         ML
##  Optimization method                           NLMINB
##  Number of model parameters                        12
##
##  Number of observations                           127
##  Number of missing patterns                         2
##
## Model Test User Model:
##
##  Test statistic                                1495.702
##  Degrees of freedom                                   15
##  P-value (Chi-square)                            0.000
##
## Model Test Baseline Model:
##
##  Test statistic                                 922.808
##  Degrees of freedom                                   15
##  P-value                                         0.000
##
## User Model versus Baseline Model:
##
##  Comparative Fit Index (CFI)                     0.000
##  Tucker-Lewis Index (TLI)                       -0.624
##
##  Robust Comparative Fit Index (CFI)              0.000
##  Robust Tucker-Lewis Index (TLI)                 0.000
##
## Loglikelihood and Information Criteria:
##
##  Loglikelihood user model (H0)               -1215.958
##  Loglikelihood unrestricted model (H1)       -489.107
##
##  Akaike (AIC)                                2461.917
##  Bayesian (BIC)                              2499.331
##  Sample-size adjusted Bayesian (SABIC)       2458.594
##
## Root Mean Square Error of Approximation:
##
##  RMSEA                                          0.847
##  90 Percent confidence interval - lower         0.806
##  90 Percent confidence interval - upper         0.888
##  P-value H_0: RMSEA <= 0.050                    0.000
##  P-value H_0: RMSEA >= 0.080                    1.000
##
##  Robust RMSEA                                   1.246
##  90 Percent confidence interval - lower         1.189
##  90 Percent confidence interval - upper         1.305
##  P-value H_0: Robust RMSEA <= 0.050             0.000
##  P-value H_0: Robust RMSEA >= 0.080             1.000
##
## Standardized Root Mean Square Residual:
##
##  SRMR                                           0.144
##
## Parameter Estimates:
##
##  Standard errors                                Standard
##  Information                                    Observed
##  Observed information based on                  Hessian
##
## Latent Variables:
##                   Estimate  Std.err  z-value  P(>|z|)   Std.lv  Std.all
##  i_gfgq =~
##    gfgq0             1.000                               0.477    0.995
##    gfgq1             1.000                               0.477    0.922
##    gfgq2             1.000                               0.477    0.798
##  s_gfgq =~
##    gfgq0             0.000                               0.000    0.000
##    gfgq1             1.000                               0.384    0.169
##    gfgq2             2.000                               0.732    0.306
##  i_area =~
##    hcesd             1.000                               0.992    0.995
##    hcesd             1.000                               0.992    0.992
##    hcesd             1.000                               0.992    0.992
##  s_area =~
##    hcesd             0.000                               0.000    0.000
##    hcesd             0.000                               0.028    0.028
##    hcesd             0.000                               0.056    0.056
##  i_area ~~
##    i_gfgq            0.000                               0.000    0.000
##    s_area            0.000                               0.000    0.000
##  s_area ~~
##    i_gfgq            0.055   0.117   0.197   0.000   0.000    0.000
##    s_gfgq            0.124   0.017   7.479   0.000   0.999    0.999
##    s_area            0.000   0.000   1.120   0.000   0.000    0.000
##    s_area            0.001   0.001   1.847   0.100   0.048    0.048
##
## R-Square:
##                   Estimate
##    gfgq0             0.995
##    gfgq1             0.851
##    gfgq2             0.992
##    hcesd             1.000
##    hcesd             1.000
##    hcesd             1.000
##    s_gfgq            0.001
##    s_area            0.152
```

```
parameterEstimates(fit_lgm_area, standardized = TRUE)
semPlot::semPaths(fit_lgm_area, what="paths", whatLabels="par", layout = "tree2",
edge.label.cex=0.8,sizeMan=8,sizeLat=8,nCharNodes=0,nCharEdges=0,rotation=2)

edge_colors_area <- ifelse(lavpulse_edges_area$pvalue < 0.05, "black", "lightgray")
edge_labels_area <- round(lavpulse_edges_area$std.all, 2)

semPaths(fit_lgm_area,
         what = "paths",
         whatLabels = as.character(edge_labels_area),
         edge.color = edge_colors_area,
         layout = "tree",
         rotation = 2,
         edge.label.cex = 0.8,
         edge.label.position = 0.6,
         sizeMan = 8,
         sizeLat = 8,
         nCharNodes = 0,
         nCharEdges = 0,
         curve = 1,
         intercepts = FALSE,
         residuals = TRUE)
```